



SINTRA
TECNOLOGIAS DIGITAIS
ECONOMIA E SOCIEDADE

Pós-Graduação em Aprendizagem Automática Aplicada

Otimização Computacional

1ª tutoria

Embeddings de palavras (Word Embeddings)

Grupo :

Eduardo Martins: 120063

João Bertulani: 120174

Professora: Rosário Laureano

Ano Letivo 2025/2026

Índice

1. Introdução.....	3
2. Técnicas de otimização.....	3
2.1 Skip-Gram.....	3
2.2 Negative Sampling.....	3
2.3 Noise Contrastive Estimation.....	4
2.4 Formulação Matemática do Problema.....	4
3. Algoritmos de Otimização.....	5
4. Metodologia.....	5
5. Resultados Esperados.....	6
6. Referências.....	7

1. Introdução

Na área da Inteligência Artificial e do Processamento de Linguagem Natural, os word embeddings constituem uma abordagem para representar palavras sob a forma de vetores numéricos. Estes vetores procuram capturar relações semânticas e sintáticas entre palavras, permitindo que modelos computacionais compreendam similaridades linguísticas e padrões contextuais.

Este projeto tem como objetivo estudar e implementar métodos de otimização aplicados à aprendizagem de embeddings de palavras. Procurar-se-á analisar o impacto de diferentes algoritmos na convergência, na estabilidade e na qualidade das representações obtidas.

Em particular, o estudo abordará o modo como técnicas como o Skip-Gram com Negative Sampling (SGNS) e a Noise Contrastive Estimation (NCE) formulam a aprendizagem destas representações como um problema de otimização.

2. Técnicas de otimização

Dado um corpus textual composto por sequências de palavras, pretende-se maximizar a probabilidade de prever palavras de contexto a partir de uma palavra central. Desta forma iremos explorar as seguintes técnicas de otimização.

2.1 Skip-Gram

Esta técnica baseia-se em escolher uma palavra central e tentar prever as palavras que aparecem à sua volta. Durante o treino, o modelo ajusta os vetores das palavras para melhorar essas previsões. Com o tempo, palavras que aparecem em contextos semelhantes passam a ter vetores semelhantes.

Por exemplo, palavras como “rei” e “rainha” ou “Portugal” e “Lisboa” acabam por ficar próximas no espaço vetorial.

2.2 Negative Sampling

O Negative Sampling é uma técnica utilizada para tornar o treino do modelo Skip-Gram mais rápido e eficiente. Em vez de comparar uma palavra com todo o vocabulário, o modelo aprende apenas com alguns exemplos positivos e negativos.

O método funciona da seguinte forma: o modelo recebe um par correto de palavras, como uma palavra central e uma palavra de contexto real, e tenta aproximar os seus embeddings. Ao mesmo tempo, recebe palavras aleatórias que não pertencem ao contexto e tenta afastar os seus embeddings.

Desta forma, o modelo aprende a distinguir relações corretas de relações incorretas, reduzindo significativamente o custo computacional do treino.

2.3 Noise Contrastive Estimation

Por último, utiliza-se a técnica de Noise Contrastive Estimation utilizada para reduzir o custo computacional no treino de modelos de linguagem e word embeddings. Em vez de calcular probabilidades para todas as palavras do vocabulário, o modelo transforma o problema num problema de classificação.

O funcionamento baseia-se em apresentar ao modelo exemplos reais do texto e exemplos de ruído gerados aleatoriamente. O objetivo do modelo é aprender a distinguir palavras corretas de palavras falsas.

Ao longo do treino, os embeddings são ajustados para aumentar a probabilidade de identificar corretamente os exemplos reais e rejeitar os exemplos de ruído.

2.4 Formulação Matemática do Problema

Para enquadrar a aprendizagem dos vetores como um problema de otimização, é necessário formular a função de objetivo que pretendemos minimizar. No contexto do Skip-Gram com Negative Sampling (SGNS), o objetivo é minimizar a função de perda L para uma palavra central ω_c e uma palavra de contexto verdadeira ω_o :

$$L = -\log \sigma(v_{\omega_o}^T \cdot v_{\omega_c}) - \sum_{i=1}^k \log \sigma(-v_{n_i}^T \cdot v_{\omega_c})$$

Onde:

- $\sigma(x) = \frac{1}{1 + e^{-x}}$ representa a função sigmoide;
- v_{w_c} é o vetor representativo da palavra central;
- v_{w_o} é o vetor representativo da palavra de contexto real;
- v_{n_i} são os vetores das k palavras de ruído (negative samples) extraídas aleatoriamente do vocabulário.

Assim, o problema de otimização foca-se em encontrar os vetores v ótimos que minimizem L ao longo de todo o corpus de treino.

3. Algoritmos de Otimização

Os algoritmos de otimização interagem diretamente sobre as técnicas de aprendizagem de embeddings, uma vez que são responsáveis pela atualização dos vetores das palavras ao longo do processo de treino. Desta forma o grupo irá explorar os seguintes algoritmos de otimização: Stochastic Gradient Descent (SGD), o RMSProp e o Adam.

O SGD realiza atualizações simples e incrementais após cada exemplo de treino, permitindo uma aprendizagem gradual dos vetores. O RMSProp adapta automaticamente a taxa de aprendizagem de acordo com o comportamento dos gradientes, proporcionando maior estabilidade durante o treino. Já o Adam combina informação de gradientes anteriores com taxas de aprendizagem adaptativas, acelerando a convergência e melhorando a eficiência do processo de otimização.

No contexto do Skip-Gram com Negative Sampling, por exemplo, o modelo procura aproximar os embeddings de palavras semanticamente relacionadas e afastar os vetores de palavras não relacionadas. O algoritmo de otimização calcula, a cada iteração, a magnitude e a direção das alterações necessárias em cada vetor, permitindo melhorar progressivamente a capacidade de representação semântica dos embeddings.

4. Metodologia

Uma vez que o tratamento de dados não é o principal foco do trabalho, iremos manter o dataset simples, de forma a manter o foco na otimização e perder menos tempo em tarefas secundárias como a limpeza e tratamento dos dados.

Desta forma, o dataset escolhido foi o Text8, que consiste num *dump* de artigos da Wikipédia já tratados e comprimidos, de forma a permitir um treino e teste rápido.

Etapa	Descrição	Ferramentas / Técnicas
1. Definição do Problema	Criar um modelo capaz de prever palavras com base no contexto.	Modelos de linguagem, Word Embeddings
2. Recolha dos dados	Selecionar um conjunto de dados adequado para treino do modelo.	Kaggle
3. Pré-processamento dos Dados	Limpeza e preparação do texto para treino.	Tokenização, normalização, remoção de caracteres especiais
4. Construção do Modelo Base	Implementar um modelo simples sem técnicas de otimização.	Skip-Gram simples

5. Treino do Modelo Base	Treinar o modelo.	TensorFlow, PyTorch
6. Aplicação de Negative Sampling	Treinar o modelo utilizando exemplos positivos e negativos.	Negative Sampling
7. Aplicação de Noise Contrastive Estimation	Transformar o problema em classificação entre exemplos reais e ruído.	Noise Contrastive Estimation
8. Treino dos Modelos Otimizados	Treinar diferentes versões do modelo com cada técnica de otimização.	TensorFlow, PyTorch
9. Avaliação dos Resultados	Avaliar o desempenho dos modelos através de métricas.	Accuracy, Loss
10. Comparação dos Modelos	Comparar modelo base e modelos otimizados.	Tabelas, gráficos comparativos
11. Análise Final	Interpretar os resultados obtidos.	Análise estatística e comparativa
12. Conclusão	Apresentar conclusões e possíveis melhorias futuras.	Jupyter Notebook

Tabela 01 - Plano de trabalho

5. Resultados Esperados

Tendo como foco um modelo capaz de prever palavras com base no contexto, pretende-se inicialmente desenvolver uma versão mais simples, sem a utilização de técnicas de otimização. Posteriormente, esse modelo será comparado com uma versão otimizada, recorrendo às técnicas de otimização anteriormente mencionadas e a diferentes algoritmos de otimização.

O objetivo é analisar e comparar os resultados obtidos entre os diferentes modelos, esperando-se que os modelos otimizados apresentem um desempenho superior. Além disso, pretende-se avaliar o impacto de cada técnica de otimização no problema em estudo, identificando quais proporcionam melhores resultados, bem como determinar qual o algoritmo de otimização mais eficiente.

Numa perspectiva puramente algorítmica, perspectiva-se que o Adam atinja a convergência da função de perda significativamente mais depressa nas épocas iniciais, dadas as suas *learning rates* dinâmicas. Por outro lado, espera-se que o SGD exiba uma descida mais ruidosa e lenta, mas possivelmente atingindo um valor final ótimo e estável. A comparação cruzada entre a convergência, estabilidade das curvas de perda e tempo de processamento ditará as conclusões finais quanto ao método mais vantajoso.

6. Referências

Yong, Y. (s.d.). *text8-zip* [Conjunto de dados]. Kaggle.

<https://www.kaggle.com/datasets/yorkyong/text8-zip>

Mahoney, M. (2006). *Large text compression benchmark* [Conjunto de dados].

<http://mattmahoney.net/dc/textdata.html>